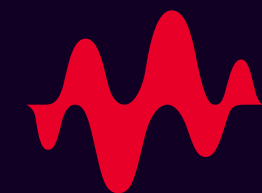




&



KEYSIGHT

Best practice sharing

Collaborative Testing Excellence: A Unified Approach

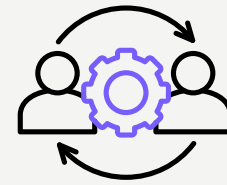
Building Effectiveness & Efficiency Through Cross-Functional Collaboration.

Quality is everyone's responsibility

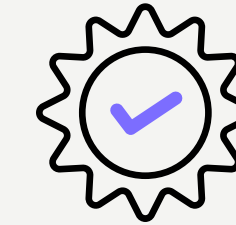
The Collaborative Framework



Shared ownership of quality.



Early and continuous collaboration.



Quality by design, not inspection.

Test Design Sessions – The Heart of Collaboration



Before feature development starts .



Developer + Tester + Product Owner (minimum) + SDET.



Collaborative brainstorming of test ideas and scenarios, determine test layers and testing depth.

Benefits

- **Shift left enabler** - Catch issues early.
- **Building testability** into design.
- **Team ownership** of Quality.
- **Improve quality culture.**
- **Finding gaps** in requirements early.
- **Building testing layers** strategically.

Test Design Session Process

From Ideas to Implementation

Preparation :

- Complex stories (5+ points) are prime candidates.
- Facilitator reviews acceptance criteria.
- Prepare breadth of test ideas (not everything!).

During Session:

- Share test ideas and gather feedback.
- Map test ideas to appropriate testing layers.
- Testing Depth - Prioritize based on risk and value.

Outcome:

- Clear testing strategy.
- Shared understanding.
- Reduced duplication.
- Execution clarity (Automation and manual testing).

Conversation Starters for Test Design

Guiding Questions for Collaborative Sessions

■ Value to Users/Business:

- What problem does this solve?
- How will we measure success?
- What analytics do we need?

■ Feature Behavior:

- What are the business rules?
- Who will use this and how?
- What's the worst/best that could happen?

■ Quality Attributes:

- Performance implications?
- Security vulnerabilities?
- Accessibility considerations?

■ Risks:

- New APIs following current patterns?
- Do we have the right expertise?
- Cross-system impacts?

■ Testability:

- How will we test this?
- Do we have the right test data?
- What new tools/learning required?



Test Coverage Models / Depth of Testing

Balancing Thoroughness with Time

■ Core Coverage:

- Acceptance Criteria testing.
- Story-level validation.
- Test designs and UX review.
- Product Manager review.
- Basic compatibility.

■ Comprehensive Coverage:

- Epic-level holistic testing.
- Negative test scenarios.
- Persona-based testing.
- Cross-team integration testing.

■ Extensive Coverage:

- Performance testing.
- Accessibility compliance.
- Security testing.
- Load and stress testing.

RACI Matrix – Clear Accountability

Who Does What in Our Collaborative Model

S.No	Activity	Product	Engineering	Testing	Devops
01	Epic, US Groomed ahead of sprint	A	C	I	-
02	Early Test Design	C	C	AR	-
03	Unit, Integration Tests	-	AR	C	I
04	E2E Test Automation	C	C	AR	I
05	Pair Testing	I	C	AR	-

AI in Testing

AI Capability Classification

■ Very High (70%+ Assistance)

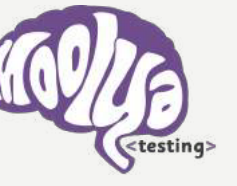
- Bug reproduction.
- Test data generation.
- Test idea generation.

■ High (50-75% Assistance)

- Risk analysis & reporting.
- Test coverage analysis.
- Production issue RCA.

■ Medium (25-50% Assistance)

- Test design sessions.
- Business requirements analysis.



Human-AI Task Classification

Five Strategic Categories

■ AI Driver:

- AI leads with human oversight.

■ Cyborg:

- Deep human-AI integration.

■ Centaur:

- Human-led with AI assistance.

■ Jagged Frontier:

- Mixed automation levels.

■ Just Me:

- Human-only tasks.

AI Implementation Approach

Key AI Tools Identified

■ NotebookLM:

- Document analysis, requirement processing.

■ Vertex AI, OpenAI:

- Pattern recognition, predictive analysis.

■ GitHub Copilot:

- Test script generation, code review.

■ Jira AI:

- Risk analysis, bug categorization.

■ Slack/Zoom AI:

- Communication automation, meeting summaries.

Key AI Tools Identified

- Structured format: Context → Expected Output → Example.
- Immediate applicability for testing activities.

Sample Prompt

"During our test design session for {feature}, suggest relevant test data that covers boundary conditions and edge cases"

Generate

"Compare {expected behavior} with {observed behavior} in production and identify potential test data needed for reproduction"

Generate

AI Implementation Approach

■ Pilot Program:

- Start with high-value, low-risk activities.

■ Tool Integration:

- Implement approved AI tools incrementally.

■ Prompt Library:

- Deploy tested prompt templates.

■ Training:

- Understanding GenAI and using GenAI as Co-worker.

■ Measurement:

- Track efficiency gains and quality impact.

Example Applications

- Automated bug reproduction using structured prompts.
- Risk analysis with AI-generated insights.
- Test design session augmentation with AI suggestions.

Example

A customer raised a tricky issue in production related to a generic Oracle error

ORA-01446: cannot select ROWID from, or sample, a view with DISTINCT, GROUP BY, etc.

"Cause: OracleDatabaseException: ORA-01446: cannot select ROWID from, or sample, a view with DISTINCT, GROUP BY, etc. What kind of data do I need to set up to reproduce the above Oracle error?"

Additional product and implementation context were provided along with the prompt

AI not only confirmed our initial suspicions but clearly pointed out how and why the error could happen. Key suggestions included:

- Creating views using UNION DISTINCT.
- Incorporating GROUP BY clauses.
- Avoiding selection of ROWID from views containing such aggregations.

Within minutes, we had a sharper hypothesis. We crafted a test view using a UNION DISTINCT clause and bingo! The error was reproducible. What could have taken hours or even days, took **less than an hour** with AI assistance.

Assessing Testing

Leading Metrics for Success

■ Team Collaboration:

- Number of collaborative test design sessions conducted.
- Participation rate across Product/Engineering/Testing.
- Pair testing sessions completed.

■ Quality of Thinking:

- Ability to ask relevant questions to stakeholders.
- Quality of test coverage models created.
- Early defect detection rate.

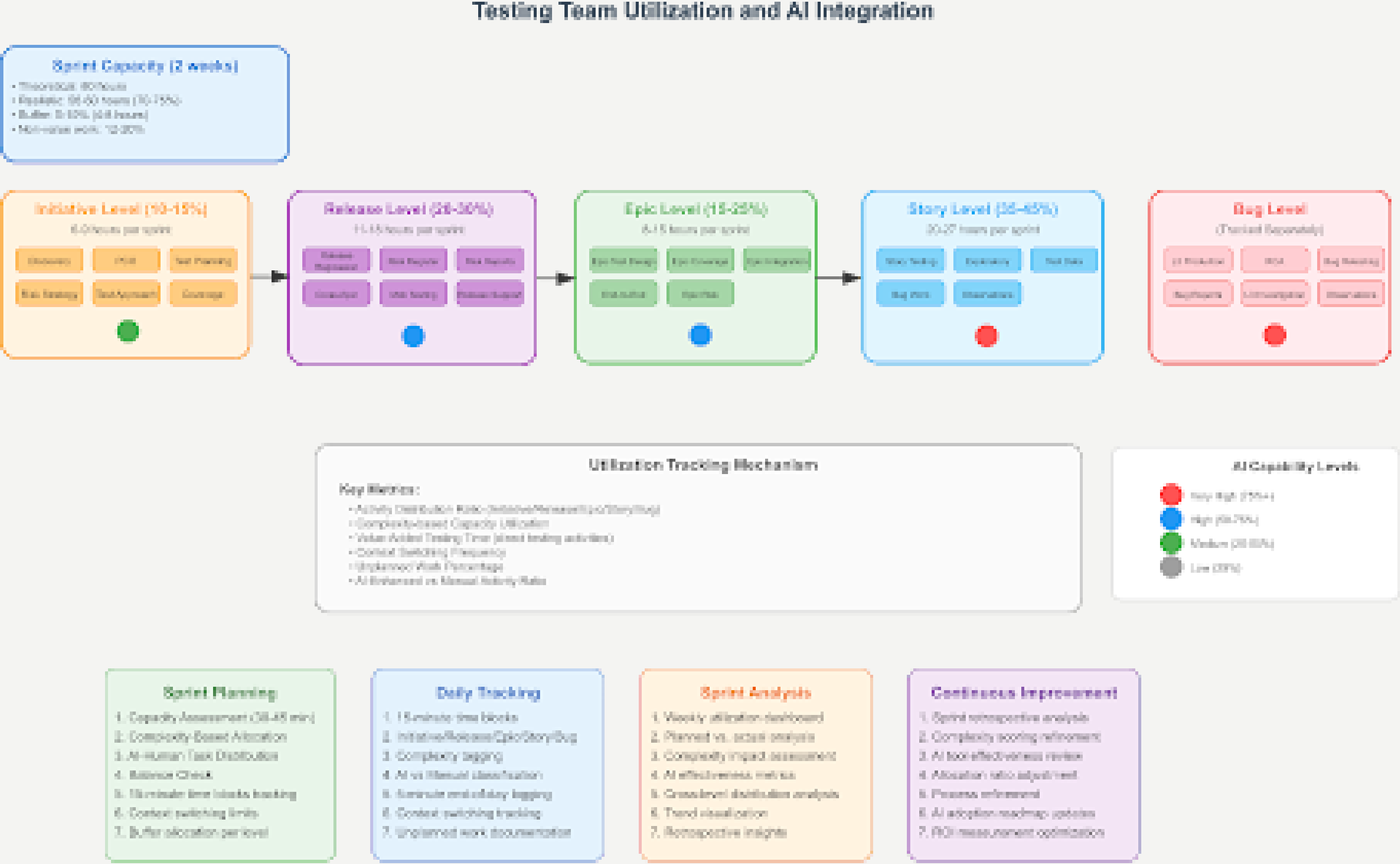
■ Process Effectiveness:

- Bug reporting clarity and actionability.
- Weekly release progress with clear coverage.
- Risk identification and mitigation.

■ Maturity Assessment:

- 0 - Not at all.
- 1 - Rarely.
- 2 - Frequently.
- 3 - Consistently.

Testing Team Utilization and AI Integration



Thank You!